Juan Manuel Copia

Madrid, Spain | jmcopia96@gmail.com | juanmacopia.github.io | linkedin.com/in/juancopia

github.com/JuanmaCopia

Summary

I am passionate about artificial intelligence and machine learning. While my PhD research focuses on program analysis and software correctness—leveraging techniques such as symbolic execution, automatic test generation, and search algorithms—I have been actively expanding my expertise in AI as a self-learner. I am particularly interested in applying machine learning to software engineering challenges and beyond. I thrive on solving complex problems through data-driven approaches and enjoy building software tools that integrate AI techniques effectively.

Experience

Research Assistant, *IMDEA Software Institute – Madrid, Spain* Dec 2020 – Present

- Designed and implemented a novel symbolic execution framework to handle complex heap-allocated inputs. Addressed a key challenge in lazy initialization, improving analysis precision and eliminating false alarms.
- Developed an efficient solver of structural constraints of partially symbolic heaps.
- Developed and applied search-based techniques to automate precondition inference.
- Integrated random test input generation with large language models to identify invalid software patches, enhancing the reliability of patch validation processes.

Research Scholarship, *Department of Computer Science – UNRC, Argentina* Oct 2019 – Oct 2020

- Built a Python-based symbolic execution engine with support for lazy initialization, enabling automated reasoning for heap-allocated objects.
- Applied the symbolic execution engine to generate automated test inputs, successfully identifying a critical bug in an educational open-source data structure.

Summer Internship, *McAfee – Córdoba, Argentina* Jan 2019 – Mar 2019

- Performed fuzz testing on internal software components, uncovering crashes and weaknesses that enhanced the software's robustness and reliability.

Student Teaching Assistant, *Department of Computer Science – UNRC, Argentina* Aug 2018 – Aug 2019

- Supported undergraduate students in 'Data Structures' and 'Algorithms', focusing on algorithm design, implementation, and problem-solving.

Education

PhD in Computer Science, *Universidad Politécnica de Madrid – Madrid, Spain* Oct 2021 – Present

Undergraduate degree in Computer Science (5 years + thesis) Mar 2015 – Dec 2020
Department of Computer Science, UNRC, Argentina

- GPA: 9.02/10.0

Undergraduate degree in Computer Science (3 years + final project) Mar 2015 – Jun 2018
Department of Computer Science, UNRC, Argentina

- GPA: 8.81/10.0

Publications

Improving Patch Correctness Analysis via Random Testing and Large Language Models May 2024

F. Molina, ***J. M. Copia***, A. Gorla.
2024 IEEE Conference on Software Testing, Verification and Validation (ICST), Toronto, ON, Canada, 2024, pp. 317-328.

doi: 10.1109/ICST60714.2024.00036

Precise Lazy Initialization for Programs with Complex Heap Inputs Oct 2023

J. M. Copia, F. Molina, N. Aguirre, M. Frias, A. Gorla, P. Ponzio.
2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE), Florence, Italy, 2023, pp. 752-762.
doi: 10.1109/ISSRE59848.2023.00080

LISSA: Lazy Initialization with Specialized Solver Aid Oct 2022

J. M. Copia, P. Ponzio, N. Aguirre, A. Gorla, M. Frias.
37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22). Association for Computing Machinery, New York, NY, USA, Article 67, 1–12.
doi: 10.1145/3551349.3556965

Use of Test Doubles in Android Testing: An In-Depth Investigation May 2022

M. Fazzini, C. Choi, ***J. M. Copia***, G. Lee, Y. Kakehi, A. Gorla, A. Orso.
2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE), Pittsburgh, PA, USA, 2022, pp. 2266-2278.
doi: 10.1145/3510003.3510175

Research Prototypes

- **Express**: A tool that automatically infers method preconditions and class invariants using search-based algorithms. It specializes in generating executable predicates (Java code) as preconditions for complex heap-allocated objects.

  Available at: github.com/JuanmaCopia/express

- **FixCheck**: A tool for improving patch correctness analysis in Java. It integrates static analysis, random testing, and large language models (LLMs) to automatically generate tests that highlight and explain potential patch inconsistencies.

  Available at: github.com/facumolina/fixcheck

- **PLI**: A symbolic execution framework built on top of Java Symbolic PathFinder (JPF). It enhances symbolic analysis for programs with complex heap-allocated inputs and intricate preconditions. PLI addresses key challenges in lazy initialization, enabling more precise and efficient symbolic reasoning.

  Available at: github.com/JuanmaCopia/spf-pli

- **SymSolve**: An efficient bounded exhaustive solver for symbolic structures with complex representation invariants. It determines the satisfiability of partially symbolic heaps with respect to structural constraints by leveraging operational specifications (e.g., repOk routines).

  Available at: github.com/JuanmaCopia/SymSolve

- **PySEAT**: A symbolic execution engine for Python programs that implements lazy initialization to represent heap-allocated objects symbolically. It automates test generation for Python codebases, achieving high coverage and uncovering potential errors in complex systems.

  Available at: github.com/JuanmaCopia/PySEAT

Public Talks

Precise Lazy Initialization for Programs with Complex Heap Inputs Apr 2024

Workshop, KLEE WORKSHOP ON SYMBOLIC EXECUTION, *Lisbon, Portugal.*

Precise Lazy Initialization for Programs with Complex Heap Inputs Oct 2023

Research Track, ISSRE 2023, *Florence, Italy.*

LISSA: Lazy Initialization with Specialized Solver Aid Oct 2022

Research Track, ASE 2022, *Oakland Center, Michigan, USA.*

LISSA: Lazy Initialization with Specialized Solver Aid Sep 2022

Oral communication, IMDEA SOFTWARE S3 SEMINAR SERIES, *Madrid, Spain.*

A Satisfiability Solver for Symbolic Structures with Complex Representation Invariants Mar 2022

Oral communication, FACAS 2022, *La Falda, Córdoba, Argentina.*

Skills

Research and Problem-Solving

- Strong analytical skills demonstrated by solving key challenges in symbolic execution and lazy initialization.
- Published research on software testing and symbolic execution at top-tier conferences.
- Self-motivated learner in AI and machine learning, actively studying topics such as deep learning, model evaluation, and data preprocessing.

Machine Learning and AI

- Experience with fundamental machine learning techniques, including classification, regression, and clustering.
- Hands-on experience with frameworks like PyTorch and scikit-learn for implementing and training models.
- Understanding of model evaluation metrics and hyperparameter tuning.
- Exploring applications of machine learning in software testing and program analysis.

Program Analysis and Software Testing

- Applied fuzz testing and symbolic execution to identify vulnerabilities and improve software reliability.
- Designed and implemented tools for analyzing software patches and programs with heap-allocated inputs.
- Developed solvers for structural constraints in partially symbolic heaps.
- Applied search-based algorithms to automate the inference of class invariants.

Programming and Tool Development

- Development of complex software tools, including symbolic execution engines, bounded exhaustive solvers, and search-based algorithms.
- Extensive experience in debugging, performance optimization, and implementing efficient algorithms.
- Building AI-driven applications and experimenting with data-driven approaches.

Collaboration and Communication

- Proven ability to work in research and development teams, collaborating with experts to create innovative tools.
- Effective in conveying technical concepts through publications, documentation, and presentations.

Programming Languages:

- Proficient in: Python, Java

Technologies:

- **Machine Learning & AI:** PyTorch, scikit-learn, NumPy, Pandas
- **Software Engineering & Testing:** Git, pytest, JUnit, Spoon (Java static analysis), Gradle
- **Formal Methods & Solvers:** Z3 (theorem prover)

Others

Spoken Languages:

- English (fluent)
- Spanish (native)
- French (fluent)